Informe de desarrollo frontend y cobertura de pruebas

PokeFoxx | TCG Market

Evaluación Parcial N.º 2 · DSY1104 · Desarrollo Full Stack II

Sebastian Carrasco · Daniel Pérez · José Venegas

24 de octubre de 2026

Repositorio público: https://github.com/SebasCarrasco19/PokeFoxx

1 Resumen del proyecto

PokeFoxx es una tienda de productos TCG que evolucionó desde HTML, CSS y JavaScript hacia una aplicación React con Vite. La segunda versión integra componentes reutilizables, rutas de tienda y administración, catálogo, categorías, ofertas, carrito, checkout simulado, órdenes, perfil y persistencia local.

Las pruebas unitarias se implementaron con Vitest y React Testing Library. El conjunto de pruebas contiene 10 archivos, 10 bloques describe y 20 tests, todos aprobados. La cobertura se obtiene con el proveedor V8.

Resultados: 20 de 20 tests aprobados, 0 fallidos. Statements y Lines: 88.78%; Branches: 71.27%; Functions: 89.28%.

2 Arquitectura

Elemento	Implementación
Interfaz	React 19, JSX y React DOM
Construcción	Vite 8
Navegación	React Router 7
Diseño	Bootstrap 5 y CSS propio
Datos	adminData.js y localStorage
Pruebas	Vitest, React Testing Library y jsdom
Cobertura	Proveedor V8; reportes de texto, JSON y HTML

El frontend actual está en frontEnd. App.jsx define las rutas; TiendaLayout y AdminLayout organizan las áreas; src/pages contiene las pantallas y src/components los componentes compartidos. adminData.js proporciona datos iniciales y localStorage conserva las modificaciones.

3 Funcionalidades de la entrega

Tienda y carrito

El usuario consulta el catálogo, los detalles, las categorías y las ofertas. Puede agregar productos, modificar cantidades, eliminar artículos y vaciar el carrito. Se calculan subtotales y total, se limita la cantidad al stock disponible y se conserva el carrito en el navegador.

Checkout invitado y con sesión

Ambos flujos muestran nombre, RUN, correo, comuna, dirección y método de pago. Sin sesión se ingresan los datos; con sesión se precargan desde la cuenta. Las modificaciones realizadas en checkout pertenecen a la compra y no actualizan el perfil.

Antes de confirmar se comprueban campos obligatorios, RUN y stock vigente. La compra genera una orden Pendiente, descuenta existencias, guarda el identificador de la última orden y vacía el carrito. El total utiliza los precios guardados en el carrito. Las páginas de resultado representan una simulación, sin cobros reales.

Cuenta y pedidos

El cliente puede registrarse, iniciar sesión, consultar y editar su perfil y revisar pedidos. Las órdenes se asocian a la cuenta por customerRun; si ese dato no existe, el historial utiliza la coincidencia de correo. Los datos del destinatario permanecen registrados en la orden.

Rol	Funciones administrativas
Cliente / invitado	Sin acceso al panel.
Vendedor	Dashboard, productos (crear, consultar, editar y eliminar), productos críticos, consulta de órdenes y perfil.
Administrador	Funciones anteriores, gestión de usuarios y categorías, reportes e historial de usuarios.

Inventario y persistencia

La vista de productos críticos incluye stock cero y productos que alcanzan o bajan del umbral configurado. Catálogo, categorías, cuentas, sesión, carrito y órdenes utilizan almacenamiento local. Los datos permanecen al recargar dentro del mismo navegador y origen.

Contenido complementario

La aplicación incluye nosotros, blog con reproductores de TikTok y contacto. El formulario de contacto de React valida sus campos y simula la confirmación del envío; no integra un servicio real de correo.

4 Inventario de pruebas automatizadas

Los archivos están en frontEnd/src/test. Cada archivo contiene un bloque describe y dos tests. La tabla resume las comprobaciones efectivamente definidas en el código.

Archivo test.jsx	Casos comprobados	Resultado
AdminDashboard	Datos calculados del Administrador; ocultación de Usuarios al Vendedor.	2 aprobados
AdminHeader	Datos de la sesión; mensaje de usuario no identificado sin sesión.	2 aprobados
AdminNav	Opciones exclusivas visibles al Administrador; ocultas al Vendedor.	2 aprobados
CartItem	Datos y subtotal; callbacks de aumentar y eliminar con el ID correcto.	2 aprobados
CheckoutForm	Datos recibidos en campos; callbacks al modificar y enviar.	2 aprobados
Contacto	Confirmación simulada y limpieza de campos; error con campos vacíos.	2 aprobados
Footer	Enlaces sin sesión; enlaces correspondientes a una sesión Cliente.	2 aprobados
Login	Rechazo de credenciales incorrectas; almacenamiento de sesión con credenciales correctas.	2 aprobados
ProductCard	Botón deshabilitado sin stock; incorporación del producto al carrito.	2 aprobados
ProductosCriticos	Inclusión de producto agotado sin umbral; mensaje cuando no existen críticos.	2 aprobados

Total: 10 archivos, 10 bloques describe y 20 tests aprobados. Las pruebas incluyen renderizado, props, contenido condicional, eventos, actualizaciones del formulario, sesión y almacenamiento local.

5 Entorno y técnicas de prueba

Vitest ejecuta la suite en jsdom. React Testing Library renderiza los componentes y consulta los elementos por texto, rol o etiqueta. setupTests.js incorpora los matchers de jest-dom. MemoryRouter proporciona el contexto de navegación a los componentes que lo requieren.

Los tests preparan datos controlados en localStorage. CartItem y CheckoutForm utilizan vi.fn() para observar callbacks; fireEvent y userEvent disparan interacciones. Estas pruebas utilizan el almacenamiento de jsdom; no definen una sustitución global propia de localStorage.

Ejemplos: CartItem comprueba un subtotal de $20.000 para dos unidades de $10.000 y valida callbacks con el ID 1. ProductCard comprueba que un producto sin stock no pueda añadirse. Login verifica que credenciales incorrectas no generen sesión.

6 Resultados de cobertura

La ejecución registrada terminó con 20 tests aprobados y 0 fallidos. El reporte de V8 presenta los siguientes resultados globales para el conjunto de archivos incluido en esa ejecución.

Métrica	Cobertura	Significado
Statements	88.78%	Sentencias ejecutadas
Branches	71.27%	Alternativas de decisión recorridas
Functions	89.28%	Funciones ejecutadas
Lines	88.78%	Líneas ejecutadas

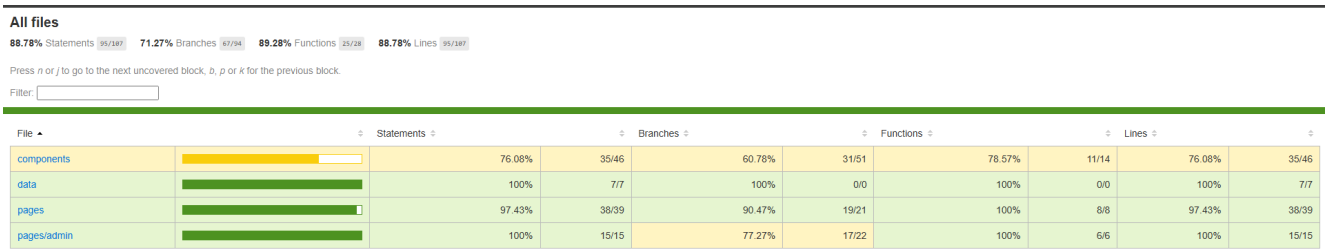


Figura 1. Resumen del reporte HTML de cobertura: 95/107 sentencias, 67/94 ramas, 25/28 funciones y 95/107 líneas ejecutadas.

6.1 Interpretación

Statements y Lines alcanzan 88.78%, mientras Functions llega a 89.28%. Branches obtiene 71.27%, el valor más bajo: existen alternativas de decisión que no fueron recorridas por los casos actuales. Las métricas representan ejecución de código y no equivalen a un porcentaje de funcionalidades correctas.

El resultado de los 20 tests es satisfactorio para las comprobaciones definidas. La cobertura no demuestra por sí sola el flujo completo de compra, la compatibilidad en todos los navegadores ni el comportamiento de todas las rutas.

6.2 Configuración del reporte

vite.config.js configura environment: jsdom, globals: true y setupFiles: ./src/setupTests.js. La cobertura utiliza provider: v8 y los reporteros text, json y html. Las exclusiones declaradas son node_modules/, src/main.jsx y los archivos **/*.d.ts.

No se define un umbral mínimo de aprobación de cobertura en esa configuración. Los porcentajes se interpretan dentro del alcance del reporte generado; no deben extrapolarse automáticamente a todos los archivos del repositorio.

6.3 Alcance funcional y alcance unitario

Las verificaciones funcionales de la entrega abarcan roles, carrito, checkout invitado y con sesión, órdenes, stock crítico, perfil y persistencia. Los 20 tests automatizados se concentran en los diez módulos de la sección 4. Por ejemplo, CheckoutForm verifica campos y eventos; no constituye una prueba automatizada de todo Checkout ni de la compra de principio a fin.

Para ampliar la suite pueden incorporarse casos de checkout completo, rechazo por stock insuficiente, persistencia del pedido, separación entre perfil y datos de entrega y acceso directo a rutas protegidas. Son ampliaciones de cobertura, no casos atribuidos a la ejecución actual.

7 Ejecución del proyecto y de las pruebas

Desde una copia del repositorio, ingresar a la carpeta frontEnd e instalar las dependencias. Los scripts están declarados en package.json.

Acción	Comando
Entrar al frontend	cd frontEnd
Instalar dependencias	npm install
Iniciar desarrollo	npm run dev
Ejecutar tests una vez	npm run test -- --run
Generar cobertura	npm run test:coverage
Compilar frontend	npm run build

test:coverage ejecuta vitest run --coverage. La terminal muestra el resultado de los tests y el resumen de cobertura. El reporte HTML se consulta en frontEnd/coverage/index.html; el reporte JSON permite conservar los datos de cobertura.

8 Evolución y entregables

Aspecto	Evaluación 1	Evaluación 2
Interfaz	Páginas HTML y JavaScript	React, componentes JSX y estado
Navegación	Enlaces entre páginas	React Router y layouts
Compra	Carrito	Checkout invitado y con sesión, resultados y órdenes
Administración	Vistas HTML	Productos, usuarios, categorías, órdenes, stock crítico y reportes
Testing	Sin suite central documentada	Vitest: 10 describe, 20 tests y cobertura V8

La documentación de la entrega se compone del ERS V2, que define requisitos y reglas, y este informe, que describe desarrollo y testing. El repositorio público contiene el frontend actualizado; los archivos HTML y el README de Evaluación 1 corresponden al antecedente histórico.

9 Conclusión

PokeFoxx incorpora los flujos de tienda y administración en React, con permisos por rol, compra simulada y persistencia local. La suite de Vitest registra 20 tests aprobados y cobertura V8 de 88.78% en Statements y Lines, 89.28% en Functions y 71.27% en Branches. La solución cumple su alcance de frontend académico; pagos, autorización de servidor y persistencia centralizada quedan fuera de esta versión.

Referencias del proyecto: repositorio público https://github.com/SebasCarrasco19/PokeFoxx; frontEnd/package.json; vite.config.js; src/App.jsx; src/pages; src/components; src/test y el ERS V2 de esta entrega.